Operator Overloading using Friend Functions:

Overloading Binary Operator using Friend Function

If you define operator function as a friend function then it will accept two arguments. Because friend functions is not a member function so it is not invoked using object of the class. Thus we need to pass two objects as an argument explicitly.
Consider following example to overload binary operator + using friend function.

```
#include <iostream>
using namespace std;
class Distance {
int feet, inch;
public:// constructor
        Distance(int a=0,int b=0)
        {
                feet = a;
                inch = b;
        }
void Print()
{
cout<<feet<<" feets ";
 if(inch!=0)
         cout<<inch<<" inches"<<endl;
}
 friend Distance operator+(Distance &,Distance &); // friend function
 friend bool operator==(Distance &,Distance &);
};
// friend function definition
Distance operator+(Distance &D1,Distance &D2)
{
```

Vector India

```
 Distance temp;
 temp.feet=D1.feet+D2.feet;
 temp.inch=D1.inch+D2.inch;
 if(temp.inch >=12)
 {
        temp.feet+=temp.inch/12;
        temp.inch+=temp.inch%12;
 }
 return temp;
}
bool operator== (Distance &D1,Distance &D2)
{
 if((D1.feet==D2.feet) && (D1.inch==D2.inch))
        return true;
return false;
}
// main function
main()
{
 Distance d1(5,8),d2(3,10),d3;
 d3=d1+d2;
 cout<<"d1:";
 d1.Print();
 cout<<"d2:";
 d2.Print();
 cout<<"d3:";
 d3.Print();
 if(d1==d2)
 cout<<"both are same:"<<endl;
 else
```

Vector India

```
 cout<<"both are not equal"<<endl;
}
```

Output:

d1: 5 feets 8 inches

d2: 3 feets 10 inches

d3: 9 feets 6 inches

both are not equal.

Overload Unary Operator using Friend Function

If you define operator function as a friend function then it will accept one argument. Because friend functions is not a member function so it is not invoked using object of the class. Thus we need to pass object as an argument explicitly.
Consider following example to overload unary operator -- using friend function.

```
#include <iostream>
using namespace std;
class Distance {
        int feet, inch;
public:// constructor
        Distance(int a=0,int b=0)
        {
                feet = a;
                inch = b;
        }
void Print()
{
        cout<<feet<<"     feets ";
         if(inch!=0)
         cout<<inch<<" inches"<<endl;
}
// pre decrement operator overloading using friend function
 friend Distance operator--(Distance &);
```


Vector India

```
//post decrement operator overloading using friend function
 friend Distance operator--(Distance &,int);
};
// friend function definition
Distance operator--(Distance &D)
{
        --D.real;
        --D.img;
        return D;
}
Distance operator--(Distance &D,int)
{
        Distance temp;
        temp.real=D.real--;
        temp.img=D.img--;
        return temp;
}
main()
{       Distance d1(5,8),d2(3,10),d3,d4;
        cout<<"before predecrement"<<endl;
        cout<<"d1:";
        d1.Print();
        cout<<"before postdecrement"<<endl;
        cout<<"d2:";
        d2.Print();
         d3=--d1;
        d4=d2--;
         cout<<"d1:";
         d1.Print();
         cout<<"d2:";
```

Vector India

```
        d2.Print();
        cout<<"d3:";
        d3.Print();
       cout<<"d4:";
       d4.Print();
}
```

Output:

```
cout<<"before predecrement"<<endl;
d1: 5 feets 8 inches
cout<<"before postdecrement"<<endl;
d2: 3 feets 10 inches
cout<<"after predecrement"<<endl;
d1: 4 feets 7 inches
cout<<"after postdecrement"<<endl;
d2: 2 feets 9 inches
d3: 4 feets 7 inches
d4: 3 feets 10 inches
```

Important points about operator overloading using friend function

- An overloaded operator friend could be declared in either private or public section of a class.
- When Binary operator overloaded function is a friend function, it takes two operands of user-defined data type.
- When Unary operator overloaded function is a friend function , it takes one operand of user-defined data type.
- Operators that cannot be overloaded using as friend function are

  = () [] ->.

Vector India